

Project 1: Build Your Own Agent Framework from Scratch

Overview & Motivation

When most developers want to build an AI agent today, they install LangChain or CrewAI and follow a tutorial. The agent works until it doesn't. And when it breaks, they have no idea why, because they never understood what was happening underneath.

This project is about building an agent framework from scratch the planning loop, memory system, and tool-calling protocol without using any agent framework library. By the end, you will understand exactly what LangChain is doing under the hood. You will be able to make informed decisions about when to use frameworks and when to avoid them. More importantly, you will be able to debug and optimize agents because you built one yourself.

This is a foundational project. Many ideas you encounter here will appear in every other AI project you ever build.

What You Will Learn

- What an "agent" actually is at a technical level
 - The ReAct (Reasoning + Acting) loop the dominant pattern in agentic AI
 - How LLMs call external tools (function calling)
 - Prompt engineering at a systems level, not just a UI level
 - Short-term vs long-term memory architectures
 - Vector embeddings and semantic search
 - How to structure a Python project as a reusable package
-

Recommended Tech Stack

--	--	--

Component	Recommended Choice	Why
Language	Python 3.11+	Largest AI/ML ecosystem, best LLM SDK support
LLM	OpenAI API (<code>gpt-4o-mini</code>)	Best-documented tool-calling API; cheap for development
Short-term memory	Python list (in-memory)	Start simple; you'll understand why you need more
Long-term memory	SQLite	No server needed; teaches persistence concepts
Semantic memory	ChromaDB	Local vector database, zero infra required
Embeddings	<code>text-embedding-3-small</code> (OpenAI)	Fast, cheap, good quality
HTTP calls	<code>openai</code> Python SDK	Handles auth, retries, response parsing
Dev environment	VS Code + <code>venv</code>	Standard; no hidden magic

Cost note: `gpt-4o-mini` costs roughly \$0.15 per million input tokens. A full agent run typically costs less than \$0.01. Budget \$3–5 for the entire project.

On choosing frameworks: You will deliberately avoid LangChain, LlamaIndex, CrewAI, and similar libraries for this project. This is not because they are bad they are useful. It is because you cannot understand what they give you until you have built what they replace.

Prerequisites Check

Before starting, make sure you are comfortable with:

- Creating a Python virtual environment (`python -m venv`) and installing packages with `pip`
- Writing a Python class with `__init__` , instance variables, and methods
- Making an HTTP API call in Python (using `requests` or any SDK)

- Reading and writing JSON
- Understanding what a function, a loop, and a conditional are

If any of these feel shaky, spend a day reviewing Python fundamentals before continuing. Everything in this project builds on these concepts.

Background Theory: What Is an LLM Agent?

A **Large Language Model (LLM)** is, at its core, a function: it takes text as input and produces text as output. Given "What is 2 + 2?", it returns "4". That is useful. That is not an agent.

An **agent** is a system that can:

1. **Observe** its environment read inputs, query tools, retrieve memories
2. **Reason** about what to do next decide which tool to call, or whether it has enough information
3. **Act** call a tool, write to memory, or produce a final response
4. **Loop** repeat this cycle until a goal is achieved

The critical insight is this: **an agent is not a smarter LLM. It is a loop around an LLM.** The LLM is the reasoning engine. The agent framework is the scaffolding that runs the loop, maintains state, and connects the LLM to the outside world.

When you hear "agentic AI," think: a while-loop with an LLM inside, connected to tools.

The ReAct Pattern

ReAct (Reasoning + Acting) is the most widely used pattern for building agents. It was introduced in a 2022 paper from Google Research and has become the foundation for most production agents.

In each iteration of the loop:

1. **Thought:** The LLM reasons about what it knows and what it still needs. (e.g., "I need to look up the current weather before answering.")
2. **Action:** The LLM selects a tool and provides arguments. (e.g., "Call `get_weather` with `city='London'` ")
3. **Observation:** Your code calls the actual tool and returns the result to the LLM.

4. Repeat until the LLM decides it has enough information to produce a **Final Answer**.

This loop is what you are building. Everything else memory, multiple agents, evaluation is built on top of this.

Phase 1: Talk to an LLM Programmatically

Goal: Make your first raw API call to an LLM and deeply understand the request and response structure.

What to Build

Write a simple Python script (not a class, not a module just a script) that:

1. Takes a hardcoded string as a prompt
2. Sends it to the OpenAI API
3. Prints the full response object (not just the text the entire thing)

Key Concept: The Messages Array

The OpenAI API does not take a single string. It takes an array of **messages**, each with a **role**. There are three roles:

- `system` : High-level instructions for the LLM. Sets its persona, constraints, and behavior for the entire conversation. The LLM reads this but cannot "respond" to it directly.
- `user` : The human's input.
- `assistant` : The LLM's previous responses.

Here is what a minimal messages array looks like conceptually:

```
[
  { "role": "system", "content": "You are a helpful assistant." },
  { "role": "user", "content": "What is the capital of France?" }
]
```

The API is **stateless**. It does not remember previous calls. Every time you send a request, you send the entire conversation history in this messages array. The API processes it and returns only the next message.

 **Think about it:** If the API is stateless and you must send the full history every time, where does that history actually live between calls? Who is responsible for maintaining it?

Write your answer down before moving on you will implement it in Phase 2.

What to Explore Before Moving On

- Look at the raw response object. What fields does it contain beyond just the text content? Note the `usage` field especially.
 - What is a **token**? It is not the same as a word. Research how the OpenAI tokenizer works. Why does the model think in tokens instead of characters or words?
 - What is the `model` parameter? What happens to cost and quality if you switch from `gpt-4o-mini` to `gpt-4o`?
-

Phase 2: Build the Conversation Manager

Goal: Turn your one-shot script into a reusable class that maintains a multi-turn conversation.

What to Build

Create a `ConversationManager` class that:

- Stores a `system_prompt` and a list of `messages` as instance variables
- Has an `add_user_message(text)` method that adds a user message to the history
- Has a `get_response()` method that sends the current history to the API and adds the assistant response to the history
- Has a `chat(user_input)` convenience method that combines the above two
- Has a `clear()` method that resets the conversation (but keeps the system prompt)
- Has a `get_history()` method that returns the message list

Key Concept: Why a Class?

You have probably written mostly procedural Python functions and scripts. As projects grow, you need a way to bundle related **state** (data that persists) with **behavior** (functions that operate on that data). A class does this.

The `ConversationManager` needs to *remember* the conversation history that is state. It needs to *add messages* and *call the API* that is behavior. Putting them in a class means you can create multiple independent conversation instances that do not interfere with each other.

🤔 **Think about it:** Your conversation history is stored in a Python list. What happens to it when your script finishes running? If a user closes the app and reopens it, will they be able to continue where they left off? Is that a problem for your use case right now? When would it become a problem?

What to Explore Before Moving On

- Hold a multi-turn conversation where a later message refers to something said earlier: "My name is Alex. Remember that." Then, three messages later: "What is my name?" Does it work? Why does it work, given that the API is stateless?
 - What is `max_tokens`? What happens if the LLM is generating a long response and hits the limit? Try it.
 - Look at the `finish_reason` field in the API response. What values can it take, and what does each mean?
-

Phase 3: Add Tool Calling

This is the most important phase. Tool calling is the mechanism that transforms a chatbot into an agent.

Background Theory: How Tool Calling Works

Modern LLMs can be given a list of **tools** structured descriptions of functions they can request to have called. The key thing to understand: **the LLM does not call the function itself**. It returns a special response that says, in structured form: "I want to call function X with arguments Y." Your code then calls the actual function and returns the result.

Think of it as a delegation protocol. The LLM is a manager who cannot open databases or browse the web directly. Instead, it issues instructions to your code (the worker), which executes them and reports back. The manager then reasons using the result.

Each tool description is a **JSON Schema** a structured format that specifies:

- The function name
- A plain-English description of what it does (the LLM reads this to decide when to use it)
- Each parameter: its name, type, and description
- Which parameters are required

The quality of your tool descriptions directly affects how well the LLM uses your tools. A

poorly written description leads to the wrong tool being called, or the right tool being called with wrong arguments.

🤔 **Think about it:** The LLM decides which tool to call based on the description you write. It does not see the code. What makes a good tool description? Think about what information the LLM would need to know in order to use the tool correctly and know when *not* to use it.

What to Build

1. Write two or three Python functions that do something useful:
 - Get the current date and time
 - Perform a calculation (area, unit conversion, etc.)
 - Read a text file from disk
 - (Optional) Query a free public API (weather, exchange rates, etc.)
2. Write a JSON schema description for each function
3. Modify your `ConversationManager` (or create a new class) to:
 - Accept a list of tool definitions
 - Include them in the API request
 - Detect when the LLM response is a tool call (vs. a regular text response)
 - Call the appropriate Python function with the provided arguments
 - Format the result and add it back to the message history
 - Call the API again so the LLM can continue reasoning

🤔 **Think about it:** The LLM might call your function with arguments you didn't expect the wrong type, a missing field, a value outside the valid range. Who is responsible for validating these arguments the LLM, or your code? What are the consequences of trusting the LLM's output blindly? Where in your code should validation happen?

🤔 **Think about it:** What should happen when a tool call fails? For example, the file doesn't exist, or the external API is down. Design this behavior before writing code. Should the agent give up? Retry? Try a different tool? Inform the user? Your answer should influence your code structure.

What to Explore Before Moving On

- Print the raw API response when the LLM wants to call a tool. What does it look like? How is it structurally different from a normal text response? Look for the `tool_calls` field.
 - What is the `tool_choice` parameter? What are its possible values? When would you force the LLM to always call a specific tool, and when would you let it decide?
 - What happens if you give the LLM a tool description for a function that does not exist? What does the LLM do?
-

Phase 4: Build the Agent Loop

Goal: Combine the conversation manager and tool calling into a loop that continues until the agent decides it has enough information to answer.

What to Build

Create an `Agent` class that:

- Accepts a `system_prompt`, a list of tools, and a `max_iterations` parameter
- Has a `run(user_input)` method that:
 - Adds the user message to history
 - Calls the LLM
 - **If** the response is a tool call: executes the tool, adds the result to history, and loops
 - **If** the response is a final text answer: returns it and stops
 - **If** `max_iterations` is exceeded: stops and returns a sensible message

Key Design Question: Loop Termination

The loop ends when the LLM produces a text response instead of a tool call. But what prevents the LLM from getting stuck repeatedly calling the same tool, or oscillating between two tools forever?

🤔 **Think about it:** You added `max_iterations` as a safety limit. But what is the right value? Too low and the agent gives up too early. Too high and a stuck agent wastes time and money. Think about the types of tasks your agent will handle. What is the longest legitimate chain of tool calls you would expect? Use that to inform your default.

🤔 **Think about it:** When the agent hits `max_iterations`, you have a few options: crash

with an error, return an empty string, return a partial result with a note, or ask the LLM to summarize what it knows so far and return that. Which is best? Why? Think from the perspective of the user of your framework.

Code Structure Check

Look at your `run()` method. List every responsibility it has what does it do?

🤔 **Think about it:** If your `run()` method does more than one clear thing, it is doing too much. Can you break it into smaller, single-purpose methods? What would you name them? Good naming is not cosmetic it is documentation.

Phase 5: Add Memory

Right now, your agent is amnesiac. Each `run()` call starts fresh, with no knowledge of previous runs. Memory is what gives an agent continuity.

Background Theory: Types of Agent Memory

Short-term memory (context window): The messages currently in the conversation. The LLM can "see" all of these simultaneously. Limited by the context window size (typically 8,000–128,000 tokens depending on the model). You already have this it is your messages list.

Long-term memory (external storage): Information stored outside the LLM, retrieved selectively when needed. Can be:

- *Verbatim storage:* Store and retrieve exact text. Good for facts, user preferences, conversation logs.
- *Semantic storage:* Convert text into a vector embedding and store the vector. Retrieve by meaning similarity, not keyword.

Episodic memory: Records of what happened in previous conversations. Answering: "What did we talk about before?"

Semantic memory: Facts and knowledge. Answering: "What do I know about this user?" or "What has this agent learned over time?"

Background Theory: Vector Embeddings

A **vector embedding** is a way of representing text as a list of numbers (a vector). For example, OpenAI's `text-embedding-3-small` model turns any piece of text into a list of 1536 floating-point numbers.

The important property: **texts that are semantically similar will have vectors that are mathematically close to each other**, measured using a metric called cosine similarity.

This means you can take a new piece of text, convert it to a vector, and find the stored memories that are most semantically similar even if they use completely different words.

Example: “I prefer dark mode in all my apps” and “The user likes interfaces without bright backgrounds” would be very close in embedding space, even though they share almost no words.

🤔 **Think about it:** If you store every single message from every conversation as a memory, you will quickly accumulate thousands of entries. When retrieving memories to help answer a new question, you fetch the top-N most similar ones. What is a good value for N? Too small and you miss relevant context. Too large and you fill your prompt with noise. What factors would you consider?

What to Build

Part A Episodic Memory (SQLite):

1. After each `run()` completes, ask the LLM to write a 2–3 sentence summary of what just happened
2. Store that summary in a SQLite database with a timestamp and the original user query
3. At the start of each `run()`, retrieve the last 3–5 summaries and inject them into the system prompt

Part B Semantic Memory (ChromaDB):

1. After each `run()`, embed key facts or takeaways from the conversation using the embeddings API
2. Store them in ChromaDB with metadata (timestamp, source, etc.)
3. At the start of each `run()`, embed the user’s new input, search ChromaDB for the top-3 most similar past memories, and inject them into the context

🤔 **Think about it:** When you inject past memories into the system prompt, you consume tokens. If the user has had 500 conversations and you inject all of them, you will exceed the context limit and the API call will fail. How would you handle this? Think about what “relevance” means in this context. Should older memories carry less weight?

🤔 **Think about it:** Some information should be remembered forever (user preferences, important facts). Other information is ephemeral (what the weather was last Tuesday). How would you distinguish between them? Could you build a mechanism to let the agent

itself decide what is worth remembering long-term?

Phase 6: Package It as a Reusable Framework

Goal: Make your agent framework something that another developer could import and use without reading your implementation.

What to Build

Reorganize your code into a proper Python package structure. A Python package is a directory with an `__init__.py` file that makes it importable. Design your package to have:

- A core `Agent` class that orchestrates the loop
- A `ToolRegistry` class for registering, validating, and dispatching tool calls
- A `Memory` base class with `SQLiteMemory` and `ChromaMemory` implementations
- A `@tool` decorator that lets a developer turn any Python function into a registered tool simply by adding the decorator above it

The test of good design: could someone use your framework to build a completely different agent say, a code review assistant by only using your public interface, without ever reading your implementation code?

🤔 **Think about it:** What should the public interface of your `Agent` class look like? Write out what you want the usage experience to be, in plain English, before touching any code. Then write code that fulfills that interface. This is called interface-first design, and it leads to better abstractions.

🤔 **Think about it:** Your `Memory` class is an abstraction over two different backends (SQLite and Chroma). What are the minimal operations that any memory backend must support? Write those as abstract methods. This is the concept of an interface or abstract base class in Python.

How to Know You're Done

Build a demo using **only your own framework** (no LangChain, no AutoGen) that satisfies all of these:

1. Can answer general questions using its own reasoning

2. Can look up current information using at least one external tool
3. Remembers facts across multiple separate `run()` sessions (test by closing and reopening your script)
4. Handles a tool failure gracefully the agent should explain the failure rather than crashing
5. Someone else can clone your repo, read your README, and use your framework to build a different agent in under an hour

If you can do all five, you have succeeded. More importantly, you now understand what every agent framework is doing under the hood.

Going Further

Once the core is working, here are extensions that push your understanding further:

- **Streaming:** Print tokens as they arrive instead of waiting for the full response. This requires understanding Server-Sent Events (SSE).
 - **Parallel tool calls:** The LLM can request multiple tool calls simultaneously. Handle this and call them concurrently using `asyncio`.
 - **Tool schemas from type hints:** Generate the JSON schema automatically from Python type hints and docstrings, eliminating manual schema writing.
 - **Compare to LangChain:** Install LangChain and build the same agent. What did LangChain give you for free? What did it make harder? Write a short comparison document.
-

Next project: [Project 2 Multi-Agent System with Failure Handling]